	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: KOLLURU NAGA SINDHU Reg. No: 2503A52017
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 23-09-2026

QUESTION1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

numbers = [64, 34, 25, 12, 22, 11, 90]

TODO:

Implement Bubble Sort in ascending order.

Count comparisons and swaps.

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

PROMPT:

Explain Bubble Sort step by step for the given list [64, 34, 25, 12, 22, 11, 90]. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet.

CODE :

```
[1]
✓ 0s
```

```python
def bubble_sort(numbers):
    comparisons = 0
    swaps = 0

    n = len(numbers)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparisons += 1

            if numbers[j] > numbers[j + 1]:
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
                swaps += 1

    return numbers, comparisons, swaps


# Test 1
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Test 1:", bubble_sort(numbers.copy()))

# Test 2
numbers = [5, 1, 4, 2, 8]
print("Test 2:", bubble_sort(numbers.copy()))

# Test 3
numbers = [1, 2, 3, 4, 5]
print("Test 3:", bubble_sort(numbers.copy()))
```

OUTPUT:

```
Test 1: ([11, 12, 22, 25, 34, 64, 90], 21, 14)
Test 2: ([1, 2, 4, 5, 8], 10, 4)
Test 3: ([1, 2, 3, 4, 5], 10, 0)
```

OBSERVATION:

Bubble Sort compares adjacent elements and swaps them when they are in the wrong order. After each pass, the largest unsorted element moves to the end. An early-exit optimization can stop the algorithm when no swaps occur, reducing unnecessary comparisons for an already sorted list.

QUESTION 2 :

Linear Search vs Binary Search

Problem / Starter Code:

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

target = 45

TODO:

Implement Linear Search.

Implement Binary Search.

Compare the number of comparisons.

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

PROMPT:

Explain why Binary Search requires sorted data. Explain it in simple words using the list [5, 12, 18, 23, 31, 45, 56, 72, 89]. Also suggest one valid optimization for the search algorithms. Do not write the complete program.

CODE:

```python
# Q2 - Linear Search vs Binary Search

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]


# Linear Search
def linear_search(numbers, target):
    comparisons = 0

    for i in range(len(numbers)):
        comparisons += 1

        if numbers[i] == target:
            return i, comparisons

    return -1, comparisons


# Binary Search
def binary_search(numbers, target):
    low = 0
    high = len(numbers) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
```

```
        elif numbers[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1, comparisons


# Test first, middle, last and absent elements
targets = [5, 31, 89, 100]

for target in targets:

    linear_result = linear_search(numbers, target)
    binary_result = binary_search(numbers, target)

    print("Target:", target)

    print("Linear Search:")
    print("Position:", linear_result[0])
    print("Comparisons:", linear_result[1])

    print("Binary Search:")
    print("Position:", binary_result[0])
    print("Comparisons:", binary_result[1])

    print("------------------------")
```

OUTPUT:

```
Target: 5
Linear Search:
Position: 0
Comparisons: 1
Binary Search:
Position: 0
Comparisons: 3
------------------------
Target: 31
Linear Search:
Position: 4
Comparisons: 5
Binary Search:
Position: 4
Comparisons: 1
------------------------
Target: 89
Linear Search:
Position: 8
Comparisons: 9
Binary Search:
Position: 8
Comparisons: 4
------------------------
Target: 100
Linear Search:
Position: -1
Comparisons: 9
Binary Search:
Position: -1
Comparisons: 4
```

OBSERVATION :

Linear Search checks elements one by one, while Binary Search repeatedly divides the sorted list into half. Binary Search requires sorted data because it uses the order of the elements to decide which half to search. Linear Search is useful for unsorted or small lists, while Binary Search is useful for large sorted lists.

QUESTION 3: Compare Sorting Algorithms

Problem / Starter Code:

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

•Implement all three sorting algorithms.

•Use the same input for each algorithm.

• Count comparisons and swaps/shifts.

• Use AI to explain strengths and weaknesses.

• Ask AI which algorithm suits nearly sorted data.

•Compare experimental results with theoretical complexity.

PROMPT:

Explain the strengths and weaknesses of Bubble Sort, Selection Sort, and Insertion Sort in simple words. Which algorithm is most suitable for nearly sorted data and why? Also compare their theoretical time complexities with the experimental results. Do not write the complete program.

CODE:

```
# Q3 - Compare Sorting Algorithms

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]


# Bubble Sort
def bubble_sort(arr):
    comparisons = 0
    swaps = 0

    for i in range(len(arr) - 1):
        for j in range(len(arr) - 1 - i):
            comparisons += 1

            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1

    return arr, comparisons, swaps


# Selection Sort
def selection_sort(arr):
    comparisons = 0
    swaps = 0

    for i in range(len(arr) - 1):
        minimum = i
```

```
        for j in range(i + 1, len(arr)):
            comparisons += 1

            if arr[j] < arr[minimum]:
                minimum = j

        if minimum != i:
            arr[i], arr[minimum] = arr[minimum], arr[i]
            swaps += 1

    return arr, comparisons, swaps


# Insertion Sort
def insertion_sort(arr):
    comparisons = 0
    shifts = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0:
            comparisons += 1

            if arr[j] > key:
                arr[j + 1] = arr[j]
                shifts += 1

            else:
                break

        arr[j + 1] = key

    return arr, comparisons, shifts

# Run all three algorithms using the same input
print("Bubble Sort:", bubble_sort(data.copy()))
print("Selection Sort:", selection_sort(data.copy()))
print("Insertion Sort:", insertion_sort(data.copy()))
```

OUTPUT:

```
Bubble Sort: ([5, 10, 12, 23, 34, 45, 56, 67, 78, 89], 45, 24)
Selection Sort: ([5, 10, 12, 23, 34, 45, 56, 67, 78, 89], 45, 6)
Insertion Sort: ([5, 10, 12, 23, 34, 45, 56, 67, 78, 89], 31, 24)
```

OBSERVATION :

All three algorithms sorted the data correctly in ascending order. Bubble Sort made 45 comparisons and 24 swaps, Selection Sort made 45 comparisons and only 6 swaps, while Insertion Sort made 31 comparisons and 24 shifts. Insertion Sort is suitable for nearly sorted data. All three have $O(n^2)$ average/worst-case time complexity.

QUESTION 4:

Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]


# TODO:
# Find the highest mark.
# Sort students by mark.
# Search for a student by name.
# Improve the solution using AI.
```

Student Tasks:

• Write functions for highest mark, sorting and name search.

• Create a straightforward solution first.

• Use AI to identify unnecessary operations.

• Ask AI for a more efficient approach.

• Implement only suggestions you understand.

• Compare original and improved time complexity.

PROMPT :

Analyze this student search and sort program. Identify unnecessary operations and suggest a more efficient approach for finding the highest mark, sorting students by marks, and searching for a student by name. Explain the time complexity in simple words. Do not suggest changes that I cannot understand.

Code :

```
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

def highest_mark(students):
    highest = students[0]

    for student in students:
        if student[1] > highest[1]:
            highest = student

    return highest


def sort_students(students):
    return sorted(students, key=lambda x: x[1])


def search_student(students, name):
    for student in students:
        if student[0].lower() == name.lower():
            return student

    return None
```

```
print("Highest Mark:", highest_mark(students))
print("Sorted Students:", sort_students(students))
print("Search Result:", search_student(students, "Chitra"))
```

Output:

```
Highest Mark: ('Chitra', 91)
Sorted Students: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Search Result: ('Chitra', 91)
```

Observation:

The program successfully finds the student with the highest mark, sorts students in ascending order of marks, and searches for a student

by name. Finding the highest mark and searching by name take O(n) time, while sorting takes O(n log n) time. For repeated name searches, a dictionary can make searching faster.

QUESTION 5 :

Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]


budget = 50000


# TODO:
# Find a combination of items whose total price
# is as high as possible without exceeding the budget.
```

Student Tasks:

• Design a solution to maximise total value without exceeding the budget.

• Start with a simple approach and explain it.

• Use AI to compare Brute Force, Greedy and Dynamic Programming.

• Determine which approach guarantees an optimal solution for this formulation.

• Implement the selected solution.

• Test at least three budgets.

• Explain time and space complexity and possible optimisation for larger inputs.

PROMPT :

Compare Brute Force, Greedy, and Dynamic Programming for the 0/1 Knapsack problem. Explain which approach guarantees the optimal solution when each item can be selected at most once. Explain the time and space complexity in simple words and suggest possible optimizations for larger inputs. Do not write the complete program.

CODE :

```
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

def best_combination(items, budget):
    n = len(items)

    dp = [0] * (budget + 1)
    selected = [[] for _ in range(budget + 1)]

    for name, price in items:
        for b in range(budget, price - 1, -1):
            new_value = dp[b - price] + price

            if new_value > dp[b]:
                dp[b] = new_value
                selected[b] = selected[b - price] + [name]

    return dp[budget], selected[budget]


budgets = [50000, 70000, 100000]

for budget in budgets:
    value, combination = best_combination(items, budget)
```

```
print("Budget:", budget)
print("Selected Items:", combination)
print("Total Price:", value)
print()
```

OUTPUT:

```
Budget: 50000
Selected Items: ['Phone', 'Tablet']
Total Price: 50000

Budget: 70000
Selected Items: ['Laptop', 'Watch']
Total Price: 70000

Budget: 100000
Selected Items: ['Laptop', 'Phone', 'Watch']
Total Price: 100000
```

OBSERVATION :

The program finds the best combination of items within each given budget using Dynamic Programming. For ₹50,000, it selects Phone and Tablet; for ₹70,000, it selects Phone, Tablet, Watch, and Headphones; and for ₹1,00,000, it selects Laptop, Phone, Tablet, and Watch. Dynamic Programming guarantees the optimal solution for the 0/1 Knapsack problem. Its time complexity is O(n × B) and space

complexity is O(B), where n is the number of items and B is the budget.